

```
In [ ]: import marimo as mo
```

Knolwedge Distillation with PyTorch

Reference: [Knowledge Distillation Tutorial](#)

Load PyTorch

```
In [ ]: import torch
import torch.nn as nn
import torch.optim as optim
import torchvision.transforms as transforms
import torchvision.datasets as datasets

# Check if the current `accelerator` <https://pytorch.org/docs/stable/torch.h
# is available, and if not, use the CPU
device = (
    torch.accelerator.current_accelerator().type if torch.accelerator.is_ava
)
print(f"Using {device} device")
```

Using cuda device

```
In [ ]: import numpy as np
import random
import os

def set_seed(seed: int = 42):
    random.seed(seed)
    np.random.seed(seed)
    os.environ["PYTHONHASHSEED"] = str(seed)
    torch.manual_seed(seed)
    torch.cuda.manual_seed(seed)
    torch.cuda.manual_seed_all(seed)  # For multi-GPU

    # Configure CUDA backends
    torch.backends.cudnn.deterministic = True
    torch.backends.cudnn.benchmark = False

    # Force use of deterministic algorithms (PyTorch 1.7+)
    torch.use_deterministic_algorithms(True)

set_seed(42)
```

Prepare Preprocessing and Load Dataset

```
In [ ]: import ssl

ssl._create_default_https_context = ssl._create_unverified_context
```

```
In [ ]: transforms_cifar = transforms.Compose(
    [
        transforms.ToTensor(),
        transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224,
    ])

# Loading the CIFAR-10 dataset
train_dataset = datasets.CIFAR10(
    root="./data", train=True, download=True, transform=transforms_cifar
)
test_dataset = datasets.CIFAR10(
    root="./data", train=False, download=True, transform=transforms_cifar
)
```

```
In [ ]: train_dataset
```

Dataset CIFAR10
Number of datapoints: 50000
Root location: ./data
Split: Train
StandardTransform
Transform: Compose(
 ToTensor()
 Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225])
)

```
In [ ]: test_dataset
```

Dataset CIFAR10
Number of datapoints: 10000
Root location: ./data
Split: Test
StandardTransform
Transform: Compose(
 ToTensor()
 Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225])
)

DataLoader

```
In [ ]: utils_code = """import random
import numpy as np
import torch
```

```

def seed_worker(worker_id):
    worker_seed = torch.initial_seed() % (2**32)
    np.random.seed(worker_seed)
    random.seed(worker_seed)
    """

with open("utils.py", "w") as f:
    f.write(utils_code)

print("Successfully wrote utils.py!")

```

Successfully wrote utils.py!

```

In [ ]: from utils import seed_worker

g = torch.Generator()
g.manual_seed(42)

train_loader = torch.utils.data.DataLoader(
    train_dataset,
    batch_size=128,
    shuffle=True,
    num_workers=2,
    worker_init_fn=seed_worker,
    generator=g,
)
test_loader = torch.utils.data.DataLoader(
    test_dataset,
    batch_size=128,
    shuffle=False,
    num_workers=2,
    worker_init_fn=seed_worker,
    generator=g,
)

```

Model classes

```

In [ ]: class DeepNN(nn.Module):
    def __init__(self, num_classes=10):
        super(DeepNN, self).__init__()
        self.features = nn.Sequential(
            nn.Conv2d(3, 128, kernel_size=3, padding=1),
            nn.ReLU(),
            nn.Conv2d(128, 64, kernel_size=3, padding=1),
            nn.ReLU(),
            nn.MaxPool2d(kernel_size=2, stride=2),
            nn.Conv2d(64, 64, kernel_size=3, padding=1),
            nn.ReLU(),
            nn.Conv2d(64, 32, kernel_size=3, padding=1),
            nn.ReLU(),
            nn.MaxPool2d(kernel_size=2, stride=2),
        )
        self.classifier = nn.Sequential(
            nn.Linear(2048, 512), nn.ReLU(), nn.Dropout(0.1), nn.Linear(512,

```

```

    )

    def forward(self, x):
        x = self.features(x)
        x = torch.flatten(x, 1)
        x = self.classifier(x)
        return x

```

```

In [ ]: class LightNN(nn.Module):
    def __init__(self, num_classes=10):
        super(LightNN, self).__init__()
        self.features = nn.Sequential(
            nn.Conv2d(3, 16, kernel_size=3, padding=1),
            nn.ReLU(),
            nn.MaxPool2d(kernel_size=2, stride=2),
            nn.Conv2d(16, 16, kernel_size=3, padding=1),
            nn.ReLU(),
            nn.MaxPool2d(kernel_size=2, stride=2),
        )
        self.classifier = nn.Sequential(
            nn.Linear(1024, 256), nn.ReLU(), nn.Dropout(0.1), nn.Linear(256,
        )

    def forward(self, x):
        x = self.features(x)
        x = torch.flatten(x, 1)
        x = self.classifier(x)
        return x

```

Utility functions

```

In [ ]: def train(model, train_loader, epochs, learning_rate, device):
    criterion = nn.CrossEntropyLoss()
    optimizer = optim.Adam(model.parameters(), lr=learning_rate)

    model.train()

    for epoch in range(epochs):
        running_loss = 0.0
        for inputs, labels in train_loader:
            # inputs: A collection of batch_size images
            # labels: A vector of dimensionality batch_size with integers de
            inputs, labels = inputs.to(device), labels.to(device)

            optimizer.zero_grad()
            outputs = model(inputs)

            # outputs: Output of the network for the collection of images. A
            # labels: The actual labels of the images. Vectors of dimensiona
            loss = criterion(outputs, labels)
            loss.backward()
            optimizer.step()

            running_loss += loss.item()

```

```
print(f"Epoch {epoch + 1}/{epochs}, Loss = {running_loss / len(train_loader)}")
```

```
In [ ]: def test(model, test_loader, device):
        model.to(device)
        model.eval()

        correct = 0
        total = 0

        with torch.no_grad():
            for inputs, labels in test_loader:
                inputs, labels = inputs.to(device), labels.to(device)

                outputs = model(inputs)
                _, predicted = torch.max(outputs.data, 1)

                total += labels.size(0)
                correct += (predicted == labels).sum().item()

        accuracy = 100 * correct / total
        print(f"Test Accuracy: {accuracy:.2f}%")
        return accuracy
```

```
[W 260707 14:32:13 compiler:352] pytest is not installed, skipping assertion rewriting
```

Train and test the model

```
In [ ]: set_seed(42)
        g.manual_seed(42)

        nn_deep = DeepNN(num_classes=10).to(device)
        train(nn_deep, train_loader, epochs=10, learning_rate=0.001, device=device)
        test_accuracy_deep = test(nn_deep, test_loader, device)
```

```
[W 260707 14:32:14 compiler:352] pytest is not installed, skipping assertion rewriting
```

```
[W 260707 14:32:14 compiler:352] pytest is not installed, skipping assertion rewriting
```

```
[W 260707 14:32:17 compiler:352] pytest is not installed, skipping assertion rewriting
```

```
[W 260707 14:32:17 compiler:352] pytest is not installed, skipping assertion rewriting
```

```
Epoch 1/10, Loss = 1.3974782518108788
```

```
[W 260707 14:32:31 compiler:352] pytest is not installed, skipping assertion rewriting
```

```
[W 260707 14:32:31 compiler:352] pytest is not installed, skipping assertion rewriting
```

```
[W 260707 14:32:34 compiler:352] pytest is not installed, skipping assertion rewriting
```

```
[W 260707 14:32:34 compiler:352] pytest is not installed, skipping assertion rewriting
```

```
Epoch 2/10, Loss = 0.9278644193773684
```

[W 260707 14:32:47 compiler:352] pytest is not installed, skipping assertion rewriting
[W 260707 14:32:47 compiler:352] pytest is not installed, skipping assertion rewriting
[W 260707 14:32:50 compiler:352] pytest is not installed, skipping assertion rewriting
[W 260707 14:32:50 compiler:352] pytest is not installed, skipping assertion rewriting

Epoch 3/10, Loss = 0.7152334691770851

[W 260707 14:33:04 compiler:352] pytest is not installed, skipping assertion rewriting
[W 260707 14:33:04 compiler:352] pytest is not installed, skipping assertion rewriting
[W 260707 14:33:07 compiler:352] pytest is not installed, skipping assertion rewriting
[W 260707 14:33:07 compiler:352] pytest is not installed, skipping assertion rewriting

Epoch 4/10, Loss = 0.5709185130760798

[W 260707 14:33:20 compiler:352] pytest is not installed, skipping assertion rewriting
[W 260707 14:33:20 compiler:352] pytest is not installed, skipping assertion rewriting
[W 260707 14:33:23 compiler:352] pytest is not installed, skipping assertion rewriting
[W 260707 14:33:23 compiler:352] pytest is not installed, skipping assertion rewriting

Epoch 5/10, Loss = 0.4526398703265373

[W 260707 14:33:36 compiler:352] pytest is not installed, skipping assertion rewriting
[W 260707 14:33:36 compiler:352] pytest is not installed, skipping assertion rewriting
[W 260707 14:33:40 compiler:352] pytest is not installed, skipping assertion rewriting
[W 260707 14:33:40 compiler:352] pytest is not installed, skipping assertion rewriting

Epoch 6/10, Loss = 0.3433163534954686

[W 260707 14:33:53 compiler:352] pytest is not installed, skipping assertion rewriting
[W 260707 14:33:53 compiler:352] pytest is not installed, skipping assertion rewriting
[W 260707 14:33:56 compiler:352] pytest is not installed, skipping assertion rewriting
[W 260707 14:33:56 compiler:352] pytest is not installed, skipping assertion rewriting

Epoch 7/10, Loss = 0.24998186305736947

[W 260707 14:34:09 compiler:352] pytest is not installed, skipping assertion rewriting
[W 260707 14:34:09 compiler:352] pytest is not installed, skipping assertion rewriting
[W 260707 14:34:12 compiler:352] pytest is not installed, skipping assertion rewriting
[W 260707 14:34:13 compiler:352] pytest is not installed, skipping assertion rewriting

Epoch 8/10, Loss = 0.19040030144783845

```
[W 260707 14:34:26 compiler:352] pytest is not installed, skipping assertion rewriting
[W 260707 14:34:26 compiler:352] pytest is not installed, skipping assertion rewriting
[W 260707 14:34:29 compiler:352] pytest is not installed, skipping assertion rewriting
[W 260707 14:34:29 compiler:352] pytest is not installed, skipping assertion rewriting
```

Epoch 9/10, Loss = 0.1632516263886486

```
[W 260707 14:34:42 compiler:352] pytest is not installed, skipping assertion rewriting
[W 260707 14:34:42 compiler:352] pytest is not installed, skipping assertion rewriting
[W 260707 14:34:45 compiler:352] pytest is not installed, skipping assertion rewriting
[W 260707 14:34:45 compiler:352] pytest is not installed, skipping assertion rewriting
```

Epoch 10/10, Loss = 0.13043279217941986

```
[W 260707 14:34:58 compiler:352] pytest is not installed, skipping assertion rewriting
[W 260707 14:34:58 compiler:352] pytest is not installed, skipping assertion rewriting
[W 260707 14:35:01 compiler:352] pytest is not installed, skipping assertion rewriting
[W 260707 14:35:01 compiler:352] pytest is not installed, skipping assertion rewriting
```

Test Accuracy: 74.75%

```
In [ ]: nn_light = LightNN(num_classes=10).to(device)
```

We instantiate one more lightweight network model to compare their performances. Back propagation is sensitive to weight initialization, so we need to make sure these two networks have the exact same initialization.

```
In [ ]: new_nn_light = LightNN(num_classes=10).to(device)
```

```
In [ ]: print("Norm of 1st layer of nn_light:", torch.norm(nn_light.features[0].weight))
# Print the norm of the first layer of the new lightweight model
print("Norm of 1st layer of new_nn_light:", torch.norm(new_nn_light.features[0].weight))
```

Norm of 1st layer of nn_light: 2.2852489948272705

Norm of 1st layer of new_nn_light: 2.337944746017456

Print the total number of parameters in each model:

```
In [ ]: total_params_deep = "{:,}".format(sum(p.numel() for p in nn_deep.parameters()))
print(f"DeepNN parameters: {total_params_deep}")
total_params_light = "{:,}".format(sum(p.numel() for p in nn_light.parameters()))
print(f"LightNN parameters: {total_params_light}")
```

DeepNN parameters: 1,186,986

LightNN parameters: 267,738

Train and test the lightweight network with cross entropy loss:

```
In [ ]: set_seed(42) # Reset seed for identical initialization
        train(nn_light, train_loader, epochs=10, learning_rate=0.001, device=device)
        test_accuracy_light_ce = test(nn_light, test_loader, device)
```

```
[W 260707 14:35:06 compiler:352] pytest is not installed, skipping assertion
rewriting
```

```
[W 260707 14:35:06 compiler:352] pytest is not installed, skipping assertion
rewriting
```

```
[W 260707 14:35:09 compiler:352] pytest is not installed, skipping assertion
rewriting
```

```
[W 260707 14:35:09 compiler:352] pytest is not installed, skipping assertion
rewriting
```

```
Epoch 1/10, Loss = 1.4699582742608113
```

```
[W 260707 14:35:17 compiler:352] pytest is not installed, skipping assertion
rewriting
```

```
[W 260707 14:35:17 compiler:352] pytest is not installed, skipping assertion
rewriting
```

```
[W 260707 14:35:21 compiler:352] pytest is not installed, skipping assertion
rewriting
```

```
[W 260707 14:35:21 compiler:352] pytest is not installed, skipping assertion
rewriting
```

```
Epoch 2/10, Loss = 1.147582862840589
```

```
[W 260707 14:35:29 compiler:352] pytest is not installed, skipping assertion
rewriting
```

```
[W 260707 14:35:29 compiler:352] pytest is not installed, skipping assertion
rewriting
```

```
[W 260707 14:35:32 compiler:352] pytest is not installed, skipping assertion
rewriting
```

```
[W 260707 14:35:32 compiler:352] pytest is not installed, skipping assertion
rewriting
```

```
Epoch 3/10, Loss = 1.000320653019049
```

```
[W 260707 14:35:41 compiler:352] pytest is not installed, skipping assertion
rewriting
```

```
[W 260707 14:35:41 compiler:352] pytest is not installed, skipping assertion
rewriting
```

```
[W 260707 14:35:44 compiler:352] pytest is not installed, skipping assertion
rewriting
```

```
[W 260707 14:35:44 compiler:352] pytest is not installed, skipping assertion
rewriting
```

```
Epoch 4/10, Loss = 0.9039682962400529
```

```
[W 260707 14:35:53 compiler:352] pytest is not installed, skipping assertion
rewriting
```

```
[W 260707 14:35:53 compiler:352] pytest is not installed, skipping assertion
rewriting
```

```
[W 260707 14:35:56 compiler:352] pytest is not installed, skipping assertion
rewriting
```

```
[W 260707 14:35:56 compiler:352] pytest is not installed, skipping assertion
rewriting
```

```
Epoch 5/10, Loss = 0.8231865765188661
```


[W 260707 14:36:04 compiler:352] pytest is not installed, skipping assertion rewriting
[W 260707 14:36:04 compiler:352] pytest is not installed, skipping assertion rewriting
[W 260707 14:36:07 compiler:352] pytest is not installed, skipping assertion rewriting
[W 260707 14:36:07 compiler:352] pytest is not installed, skipping assertion rewriting

Epoch 6/10, Loss = 0.7540609155164655

[W 260707 14:36:16 compiler:352] pytest is not installed, skipping assertion rewriting
[W 260707 14:36:16 compiler:352] pytest is not installed, skipping assertion rewriting
[W 260707 14:36:20 compiler:352] pytest is not installed, skipping assertion rewriting
[W 260707 14:36:20 compiler:352] pytest is not installed, skipping assertion rewriting

Epoch 7/10, Loss = 0.6953719047176868

[W 260707 14:36:28 compiler:352] pytest is not installed, skipping assertion rewriting
[W 260707 14:36:28 compiler:352] pytest is not installed, skipping assertion rewriting
[W 260707 14:36:31 compiler:352] pytest is not installed, skipping assertion rewriting
[W 260707 14:36:31 compiler:352] pytest is not installed, skipping assertion rewriting

Epoch 8/10, Loss = 0.6393959798928722

[W 260707 14:36:40 compiler:352] pytest is not installed, skipping assertion rewriting
[W 260707 14:36:40 compiler:352] pytest is not installed, skipping assertion rewriting
[W 260707 14:36:43 compiler:352] pytest is not installed, skipping assertion rewriting
[W 260707 14:36:43 compiler:352] pytest is not installed, skipping assertion rewriting

Epoch 9/10, Loss = 0.5909901214834979

[W 260707 14:36:51 compiler:352] pytest is not installed, skipping assertion rewriting
[W 260707 14:36:51 compiler:352] pytest is not installed, skipping assertion rewriting
[W 260707 14:36:54 compiler:352] pytest is not installed, skipping assertion rewriting
[W 260707 14:36:54 compiler:352] pytest is not installed, skipping assertion rewriting

Epoch 10/10, Loss = 0.5399060561833784

[W 260707 14:37:02 compiler:352] pytest is not installed, skipping assertion rewriting
[W 260707 14:37:03 compiler:352] pytest is not installed, skipping assertion rewriting
[W 260707 14:37:05 compiler:352] pytest is not installed, skipping assertion rewriting
[W 260707 14:37:05 compiler:352] pytest is not installed, skipping assertion rewriting

Test Accuracy: 71.12%

```
In [ ]: print(f"Teacher accuracy: {test_accuracy_deep:.2f}%")
        print(f"Student accuracy: {test_accuracy_light_ce:.2f}%")
```

Teacher accuracy: 74.75%

Student accuracy: 71.12%

Knowledge distillation

Now let's try to improve the test accuracy of the student network by incorporating the teacher. Knowledge distillation is a straightforward technique to achieve this, based on the fact that both networks output a probability distribution over our classes.

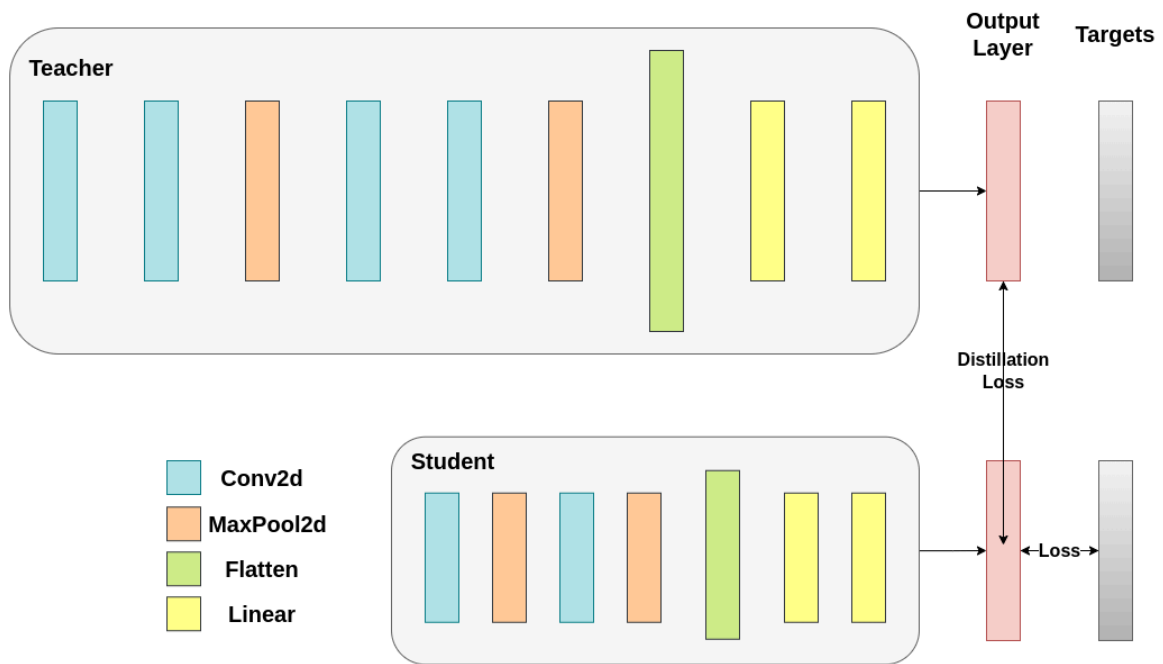
Therefore, the two networks share the same number of output neurons. The method works by incorporating an additional loss into the traditional cross entropy loss, which is based on the softmax output of the teacher network. **The assumption is that the output activations of a properly trained teacher network carry additional information that can be leveraged by a student network during training.** The original work suggests that utilizing ratios of smaller probabilities in the soft targets can help achieve the underlying objective of deep neural networks, which is to create a similarity structure over the data where similar objects are mapped closer together.

For example, in CIFAR-10, a truck could be mistaken for an automobile or airplane, if its wheels are present, but it is less likely to be mistaken for a dog. Therefore, it makes sense to assume that valuable information resides not only in the top prediction of a properly trained model but in the entire output distribution. However, cross entropy alone does not sufficiently exploit this information as the activations for non-predicted classes tend to be so small that propagated gradients do not meaningfully change the weights to construct this desirable vector space.

As we continue defining our first helper function that introduces a teacher-student dynamic, we need to include a few extra parameters:

- `T` : Temperature that controls the smoothness of the output distributions.
- `soft_target_loss_weight` : A weight assigned to the extra objective we're about to include.
- `ce_loss_weight` : A weight assigned to cross-entropy. Tuning these weights pushes the network towards optimizing for either objective.

```
In [ ]: mo.image(
        src="https://docs.pytorch.org/tutorials/_static/img/knowledge_distillati
    )
```



```
In [ ]: def train_knowledge_distillation(
    teacher,
    student,
    train_loader,
    epochs,
    learning_rate,
    T,
    soft_target_loss_weight,
    ce_loss_weight,
    device,
):
    cross_entropy_loss = nn.CrossEntropyLoss()
    optimizer = optim.Adam(student.parameters(), lr=learning_rate)

    teacher.eval()
    student.train()

    for epoch in range(epochs):
        running_loss = 0.0
        for inputs, labels in train_loader:
            inputs, labels = inputs.to(device), labels.to(device)

            optimizer.zero_grad()

            with torch.no_grad():
                teacher_logits = teacher(inputs)

            student_logits = student(inputs)

            # soften the student logits by first applying softmax, then log
            soft_targets = nn.functional.softmax(teacher_logits / T, dim=-1)
            soft_prob = nn.functional.log_softmax(student_logits / T, dim=-1)

            # Calculate the true label loss
```

```

        label_loss = cross_entropy_loss(student_logits, labels)

        # Calculate the soft targets loss. Scaled by T**2 as suggested by
        soft_targets_loss = (
            torch.sum(soft_targets * (soft_targets.log() - soft_prob))
            / soft_prob.size()[0]
            * (T**2)
        )

        # Calculate the true label loss
        label_loss = cross_entropy_loss(student_logits, labels)

        # Weighted sum of the two losses
        loss = soft_target_loss_weight * soft_targets_loss + ce_loss_weight * label_loss

        loss.backward()
        optimizer.step()

        running_loss += loss.item()

    print(f"Epoch {epoch + 1}/{epochs}, Loss: {running_loss / len(train_loader)}")

```

```

In [ ]: train_knowledge_distillation(
        teacher=nn_deep,
        student=new_nn_light,
        train_loader=train_loader,
        epochs=10,
        learning_rate=0.001,
        T=2,
        soft_target_loss_weight=0.25,
        ce_loss_weight=0.75,
        device=device,
    )
    test_accuracy_light_ce_and_kd = test(new_nn_light, test_loader, device)

    # Compare the student test accuracy with and without the teacher, after distillation
    print(f"Teacher accuracy: {test_accuracy_deep:.2f}%")
    print(f"Student accuracy without teacher: {test_accuracy_light_ce:.2f}%")
    print(f"Student accuracy with CE + KD: {test_accuracy_light_ce_and_kd:.2f}%")

```

```

[W 260707 14:37:10 compiler:352] pytest is not installed, skipping assertion rewriting
[W 260707 14:37:10 compiler:352] pytest is not installed, skipping assertion rewriting
[W 260707 14:37:13 compiler:352] pytest is not installed, skipping assertion rewriting
[W 260707 14:37:13 compiler:352] pytest is not installed, skipping assertion rewriting
Epoch 1/10, Loss: 2.426335568623165

```

[W 260707 14:37:22 compiler:352] pytest is not installed, skipping assertion rewriting
[W 260707 14:37:22 compiler:352] pytest is not installed, skipping assertion rewriting
[W 260707 14:37:25 compiler:352] pytest is not installed, skipping assertion rewriting
[W 260707 14:37:25 compiler:352] pytest is not installed, skipping assertion rewriting

Epoch 2/10, Loss: 1.8506279753906953

[W 260707 14:37:34 compiler:352] pytest is not installed, skipping assertion rewriting
[W 260707 14:37:34 compiler:352] pytest is not installed, skipping assertion rewriting
[W 260707 14:37:37 compiler:352] pytest is not installed, skipping assertion rewriting
[W 260707 14:37:37 compiler:352] pytest is not installed, skipping assertion rewriting

Epoch 3/10, Loss: 1.598138683592267

[W 260707 14:37:46 compiler:352] pytest is not installed, skipping assertion rewriting
[W 260707 14:37:46 compiler:352] pytest is not installed, skipping assertion rewriting
[W 260707 14:37:49 compiler:352] pytest is not installed, skipping assertion rewriting
[W 260707 14:37:49 compiler:352] pytest is not installed, skipping assertion rewriting

Epoch 4/10, Loss: 1.4386574447612324

[W 260707 14:37:59 compiler:352] pytest is not installed, skipping assertion rewriting
[W 260707 14:37:59 compiler:352] pytest is not installed, skipping assertion rewriting
[W 260707 14:38:02 compiler:352] pytest is not installed, skipping assertion rewriting
[W 260707 14:38:02 compiler:352] pytest is not installed, skipping assertion rewriting

Epoch 5/10, Loss: 1.3191257992668834

[W 260707 14:38:11 compiler:352] pytest is not installed, skipping assertion rewriting
[W 260707 14:38:11 compiler:352] pytest is not installed, skipping assertion rewriting
[W 260707 14:38:14 compiler:352] pytest is not installed, skipping assertion rewriting
[W 260707 14:38:14 compiler:352] pytest is not installed, skipping assertion rewriting

Epoch 6/10, Loss: 1.2159982303829144

[W 260707 14:38:23 compiler:352] pytest is not installed, skipping assertion rewriting
[W 260707 14:38:23 compiler:352] pytest is not installed, skipping assertion rewriting
[W 260707 14:38:26 compiler:352] pytest is not installed, skipping assertion rewriting
[W 260707 14:38:26 compiler:352] pytest is not installed, skipping assertion rewriting

Epoch 7/10, Loss: 1.1227675360791824

```
[W 260707 14:38:35 compiler:352] pytest is not installed, skipping assertion rewriting
[W 260707 14:38:35 compiler:352] pytest is not installed, skipping assertion rewriting
[W 260707 14:38:38 compiler:352] pytest is not installed, skipping assertion rewriting
[W 260707 14:38:39 compiler:352] pytest is not installed, skipping assertion rewriting
```

Epoch 8/10, Loss: 1.0356456856898335

```
[W 260707 14:38:47 compiler:352] pytest is not installed, skipping assertion rewriting
[W 260707 14:38:47 compiler:352] pytest is not installed, skipping assertion rewriting
[W 260707 14:38:51 compiler:352] pytest is not installed, skipping assertion rewriting
[W 260707 14:38:51 compiler:352] pytest is not installed, skipping assertion rewriting
```

Epoch 9/10, Loss: 0.9564500638591055

```
[W 260707 14:39:00 compiler:352] pytest is not installed, skipping assertion rewriting
[W 260707 14:39:00 compiler:352] pytest is not installed, skipping assertion rewriting
[W 260707 14:39:03 compiler:352] pytest is not installed, skipping assertion rewriting
[W 260707 14:39:03 compiler:352] pytest is not installed, skipping assertion rewriting
```

Epoch 10/10, Loss: 0.8875487555018471

```
[W 260707 14:39:11 compiler:352] pytest is not installed, skipping assertion rewriting
[W 260707 14:39:11 compiler:352] pytest is not installed, skipping assertion rewriting
[W 260707 14:39:14 compiler:352] pytest is not installed, skipping assertion rewriting
[W 260707 14:39:14 compiler:352] pytest is not installed, skipping assertion rewriting
```

Test Accuracy: 71.17%

Teacher accuracy: 74.75%

Student accuracy without teacher: 71.12%

Student accuracy with CE + KD: 71.17%

Cosine loss minimization run

Feel free to play around with the temperature parameter that controls the softness of the softmax function and the loss coefficients. In neural networks, it is easy to include additional loss functions to the main objectives to achieve goals like better generalization. Let's try including an objective for the student, but now let's focus on their hidden states rather than their output layers. Our goal is to convey information from the teacher's representation to the student by including a naive loss function, whose minimization implies that the flattened vectors that are subsequently passed to the classifiers have become more similar as the loss decreases. Of course, the teacher does not update its weights, so the minimization depends only on the student's

weights. The rationale behind this method is that we are operating under the assumption that the teacher model has a better internal representation that is unlikely to be achieved by the student without external intervention, therefore we artificially push the student to mimic the internal representation of the teacher. Whether or not this will end up helping the student is not straightforward, though, because pushing the lightweight network to reach this point could be a good thing, assuming that we have found an internal representation that leads to better test accuracy, but it could also be harmful because the networks have different architectures and the student does not have the same learning capacity as the teacher. In other words, there is no reason for these two vectors, the student's and the teacher's to match per component. The student could reach an internal representation that is a permutation of the teacher's and it would be just as efficient. Nonetheless, we can still run a quick experiment to figure out the impact of this method. We will be using the CosineEmbeddingLoss which is given by the following formula:

$$\text{loss}(x, y) = \begin{cases} 1 - \cos(x_1, x_2), & \text{if } y = 1 \\ \max(0, \cos(x_1, x_2) - \text{margin}), & \text{if } y = -1 \end{cases}$$

Obviously, there is one thing that we need to resolve first. When we applied distillation to the output layer we mentioned that both networks have the same number of neurons, equal to the number of classes. However, this is not the case for the layer following our convolutional layers. Here, the teacher has more neurons than the student after the flattening of the final convolutional layer. Our loss function accepts two vectors of equal dimensionality as inputs, therefore we need to somehow match them. We will solve this by including an average pooling layer after the teacher's convolutional layer to reduce its dimensionality to match that of the student.

To proceed, we will modify our model classes, or create new ones. Now, the forward function returns not only the logits of the network but also the flattened hidden representation after the convolutional layer. We include the aforementioned pooling for the modified teacher.

```
In [ ]: class ModifiedDeepNNCosine(nn.Module):
        def __init__(self, num_classes=10):
            super(ModifiedDeepNNCosine, self).__init__()
            self.features = nn.Sequential(
                nn.Conv2d(3, 128, kernel_size=3, padding=1),
                nn.ReLU(),
                nn.Conv2d(128, 64, kernel_size=3, padding=1),
                nn.ReLU(),
                nn.MaxPool2d(kernel_size=2, stride=2),
                nn.Conv2d(64, 64, kernel_size=3, padding=1),
                nn.ReLU(),
                nn.Conv2d(64, 32, kernel_size=3, padding=1),
                nn.ReLU(),
```

```

        nn.MaxPool2d(kernel_size=2, stride=2),
    )
    self.classifier = nn.Sequential(
        nn.Linear(2048, 512), nn.ReLU(), nn.Dropout(0.1), nn.Linear(512,
    )

    def forward(self, x):
        x = self.features(x)
        flattened_conv_output = torch.flatten(x, 1)
        x = self.classifier(flattened_conv_output)
        flattened_conv_output_after_pooling = torch.nn.functional.avg_pool1d(
            flattened_conv_output, 2
        )
        return x, flattened_conv_output_after_pooling

```

```

In [ ]: class ModifiedLightNNCosine(nn.Module):
    def __init__(self, num_classes=10):
        super(ModifiedLightNNCosine, self).__init__()
        self.features = nn.Sequential(
            nn.Conv2d(3, 16, kernel_size=3, padding=1),
            nn.ReLU(),
            nn.MaxPool2d(kernel_size=2, stride=2),
            nn.Conv2d(16, 16, kernel_size=3, padding=1),
            nn.ReLU(),
            nn.MaxPool2d(kernel_size=2, stride=2),
        )
        self.classifier = nn.Sequential(
            nn.Linear(1024, 256), nn.ReLU(), nn.Dropout(0.1), nn.Linear(256,
        )

    def forward(self, x):
        x = self.features(x)
        flattened_conv_output = torch.flatten(x, 1)
        x = self.classifier(flattened_conv_output)
        return x, flattened_conv_output

```

```

In [ ]: modified_nn_deep = ModifiedDeepNNCosine(num_classes=10).to(device)
modified_nn_deep.load_state_dict(nn_deep.state_dict(), strict=True)

# Once again ensure the norm of the first layer is the same for both networks
print("Norm of 1st layer for deep_nn:", torch.norm(nn_deep.features[0].weight))
print(
    "Norm of 1st layer for modified_deep_nn:",
    torch.norm(modified_nn_deep.features[0].weight).item(),
)

```

```

Norm of 1st layer for deep_nn: 7.526254653930664
Norm of 1st layer for modified_deep_nn: 7.526254653930664

```

```

In [ ]: modified_nn_light = ModifiedLightNNCosine(num_classes=10).to(device)
print("Norm of 1st layer:", torch.norm(modified_nn_light.features[0].weight))

```

```

Norm of 1st layer: 2.2852489948272705

```

Naturally, we need to change the train loop because now the model returns a tuple (logits, hidden_representation). Using a sample input tensor we can print

their shapes.

```
In [ ]: sample_input = torch.randn(128, 3, 32, 32).to(device)

# Pass the input through the student
logits, hidden_representation = modified_nn_light(sample_input)

# Print the shapes of the tensors
print(f"Student logits shape: {logits.shape}")
print(f"Student hidden representation shape: {hidden_representation.shape}")

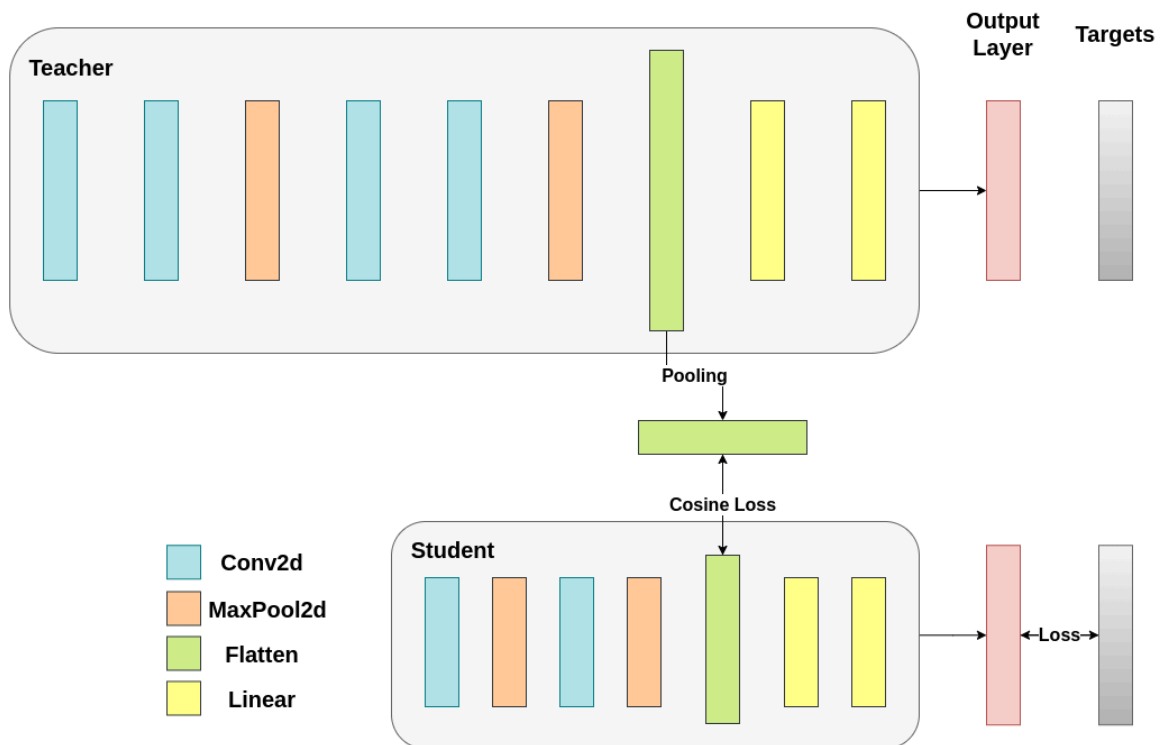
# Pass the input through the teacher
logits, hidden_representation = modified_nn_deep(sample_input)

# Print the shapes of the tensors
print(f"Teacher logits shape: {logits.shape}")
print(f"Teacher hidden representation shape: {hidden_representation.shape}")
```

```
Student logits shape: torch.Size([128, 10])
Student hidden representation shape: torch.Size([128, 1024])
Teacher logits shape: torch.Size([128, 10])
Teacher hidden representation shape: torch.Size([128, 1024])
```

In our case, `hidden_representation_size` is 1024. This is the flattened feature map of the final convolutional layer of the student and as you can see, it is the input for its classifier. It is 1024 for the teacher too, because we made it so with `avg_pool1d` from 2048. The loss applied here only affects the weights of the student prior to the loss calculation. In other words, it does not affect the classifier of the student.

```
In [ ]: mo.image(
    src="https://docs.pytorch.org/tutorials/_static/img/knowledge_distillati
    )
```



```
In [ ]: def train_cosine_loss(
    teacher,
    student,
    train_loader,
    epochs,
    learning_rate,
    hidden_rep_loss_weight,
    ce_loss_weight,
    device,
):
    cross_entropy_loss = nn.CrossEntropyLoss()
    cosine_loss = nn.CosineEmbeddingLoss()
    optimizer = optim.Adam(student.parameters(), lr=learning_rate)

    teacher.to(device)
    student.to(device)
    teacher.eval()
    student.train()

    for epoch in range(epochs):
        running_loss = 0.0
        for inputs, labels in train_loader:
            inputs, labels = inputs.to(device), labels.to(device)

            optimizer.zero_grad()

            # Forward pass with the teacher model and keep only the hidden representation
            with torch.no_grad():
                _, teacher_hidden_representation = teacher(inputs)

            # Forward pass with the student model
```

```

        student_logits, student_hidden_representation = student(inputs)

        # Calculate the cosine loss.
        # Target is a vector of ones.
        # From the loss formula above we can see that is the case where
        hidden_rep_loss = cosine_loss(
            student_hidden_representation,
            teacher_hidden_representation,
            target=torch.ones(inputs.size(0)).to(device),
        )

        # Calculate the true label loss
        label_loss = cross_entropy_loss(student_logits, labels)

        # Weighted sum of the two losses
        loss = hidden_rep_loss_weight * hidden_rep_loss + ce_loss_weight

        loss.backward()
        optimizer.step()

        running_loss += loss.item()

    print(f"Epoch {epoch + 1}/{epochs}, Loss: {running_loss / len(train_

```

```

In [ ]: def test_multiple_outputs(model, test_loader, device):
        model.to(device)
        model.eval()

        correct = 0
        total = 0

        with torch.no_grad():
            for inputs, labels in test_loader:
                inputs, labels = inputs.to(device), labels.to(device)

                output, _ = model(inputs) # Disregard the second tensor of the
                _, predicted = torch.max(output.data, 1)

                total += labels.size(0)
                correct += (predicted == labels).sum().item()

        accuracy = 100 * correct / total
        print(f"Test accuracy: {accuracy:.2f}%")
        return accuracy

```

[W 260707 14:39:18 compiler:352] pytest is not installed, skipping assertion rewriting

```

In [ ]: set_seed(42) # Reset seed for identical initialization
        train_cosine_loss(
            teacher=modified_nn_deep,
            student=modified_nn_light,
            train_loader=train_loader,
            epochs=10,
            learning_rate=0.001,
            hidden_rep_loss_weight=0.25,

```

```
    ce_loss_weight=0.75,  
    device=device,  
)  
test_accuracy_light_ce_and_cosine_loss = test_multiple_outputs(  
    modified_nn_light, test_loader, device  
)
```

[W 260707 14:39:19 compiler:352] pytest is not installed, skipping assertion rewriting

[W 260707 14:39:19 compiler:352] pytest is not installed, skipping assertion rewriting

[W 260707 14:39:22 compiler:352] pytest is not installed, skipping assertion rewriting

[W 260707 14:39:22 compiler:352] pytest is not installed, skipping assertion rewriting

Epoch 1/10, Loss: 1.301514346886169

[W 260707 14:39:32 compiler:352] pytest is not installed, skipping assertion rewriting

[W 260707 14:39:32 compiler:352] pytest is not installed, skipping assertion rewriting

[W 260707 14:39:35 compiler:352] pytest is not installed, skipping assertion rewriting

[W 260707 14:39:35 compiler:352] pytest is not installed, skipping assertion rewriting

Epoch 2/10, Loss: 1.0561232839703865

[W 260707 14:39:44 compiler:352] pytest is not installed, skipping assertion rewriting

[W 260707 14:39:44 compiler:352] pytest is not installed, skipping assertion rewriting

[W 260707 14:39:48 compiler:352] pytest is not installed, skipping assertion rewriting

[W 260707 14:39:48 compiler:352] pytest is not installed, skipping assertion rewriting

Epoch 3/10, Loss: 0.9531420345806405

[W 260707 14:39:57 compiler:352] pytest is not installed, skipping assertion rewriting

[W 260707 14:39:57 compiler:352] pytest is not installed, skipping assertion rewriting

[W 260707 14:40:01 compiler:352] pytest is not installed, skipping assertion rewriting

[W 260707 14:40:01 compiler:352] pytest is not installed, skipping assertion rewriting

Epoch 4/10, Loss: 0.8764619877576219

[W 260707 14:40:10 compiler:352] pytest is not installed, skipping assertion rewriting

[W 260707 14:40:10 compiler:352] pytest is not installed, skipping assertion rewriting

[W 260707 14:40:13 compiler:352] pytest is not installed, skipping assertion rewriting

[W 260707 14:40:13 compiler:352] pytest is not installed, skipping assertion rewriting

Epoch 5/10, Loss: 0.8264229427213254

[W 260707 14:40:23 compiler:352] pytest is not installed, skipping assertion rewriting
[W 260707 14:40:23 compiler:352] pytest is not installed, skipping assertion rewriting
[W 260707 14:40:27 compiler:352] pytest is not installed, skipping assertion rewriting
[W 260707 14:40:27 compiler:352] pytest is not installed, skipping assertion rewriting

Epoch 6/10, Loss: 0.778819742867404

[W 260707 14:42:24 compiler:352] pytest is not installed, skipping assertion rewriting
[W 260707 14:42:24 compiler:352] pytest is not installed, skipping assertion rewriting
[W 260707 14:42:28 compiler:352] pytest is not installed, skipping assertion rewriting
[W 260707 14:42:28 compiler:352] pytest is not installed, skipping assertion rewriting

Epoch 7/10, Loss: 0.7407438739791246

[W 260707 14:42:37 compiler:352] pytest is not installed, skipping assertion rewriting
[W 260707 14:42:37 compiler:352] pytest is not installed, skipping assertion rewriting
[W 260707 14:42:41 compiler:352] pytest is not installed, skipping assertion rewriting
[W 260707 14:42:41 compiler:352] pytest is not installed, skipping assertion rewriting

Epoch 8/10, Loss: 0.7084508797396785

[W 260707 14:42:50 compiler:352] pytest is not installed, skipping assertion rewriting
[W 260707 14:42:50 compiler:352] pytest is not installed, skipping assertion rewriting
[W 260707 14:42:54 compiler:352] pytest is not installed, skipping assertion rewriting
[W 260707 14:42:54 compiler:352] pytest is not installed, skipping assertion rewriting

Epoch 9/10, Loss: 0.6773196429090427

[W 260707 14:43:04 compiler:352] pytest is not installed, skipping assertion rewriting
[W 260707 14:43:04 compiler:352] pytest is not installed, skipping assertion rewriting
[W 260707 14:43:07 compiler:352] pytest is not installed, skipping assertion rewriting
[W 260707 14:43:07 compiler:352] pytest is not installed, skipping assertion rewriting

Epoch 10/10, Loss: 0.6487434914959666

[W 260707 14:43:17 compiler:352] pytest is not installed, skipping assertion rewriting
[W 260707 14:43:17 compiler:352] pytest is not installed, skipping assertion rewriting
[W 260707 14:43:19 compiler:352] pytest is not installed, skipping assertion rewriting
[W 260707 14:43:19 compiler:352] pytest is not installed, skipping assertion rewriting

Test accuracy: 71.46%

Intermediate regressor run

Our naive minimization does not guarantee better results for several reasons, one being the dimensionality of the vectors. Cosine similarity generally works better than Euclidean distance for vectors of higher dimensionality, but we were dealing with vectors with 1024 components each, so it is much harder to extract meaningful similarities. Furthermore, as we mentioned, pushing towards a match of the hidden representation of the teacher and the student is not supported by theory. There are no good reasons why we should be aiming for a 1:1 match of these vectors.

We will provide a final example of training intervention by including an extra network called regressor. The objective is to first extract the feature map of the teacher after a convolutional layer, then extract a feature map of the student after a convolutional layer, and finally try to match these maps. However, this time, we will introduce a regressor between the networks to facilitate the matching process. The regressor will be trainable and ideally will do a better job than our naive cosine loss minimization scheme. Its main job is to match the dimensionality of these feature maps so that we can properly define a loss function between the teacher and the student. Defining such a loss function provides a teaching “path,” which is basically a flow to back-propagate gradients that will change the student's weights. Focusing on the output of the convolutional layers right before each classifier for our original networks, we have the following shapes:

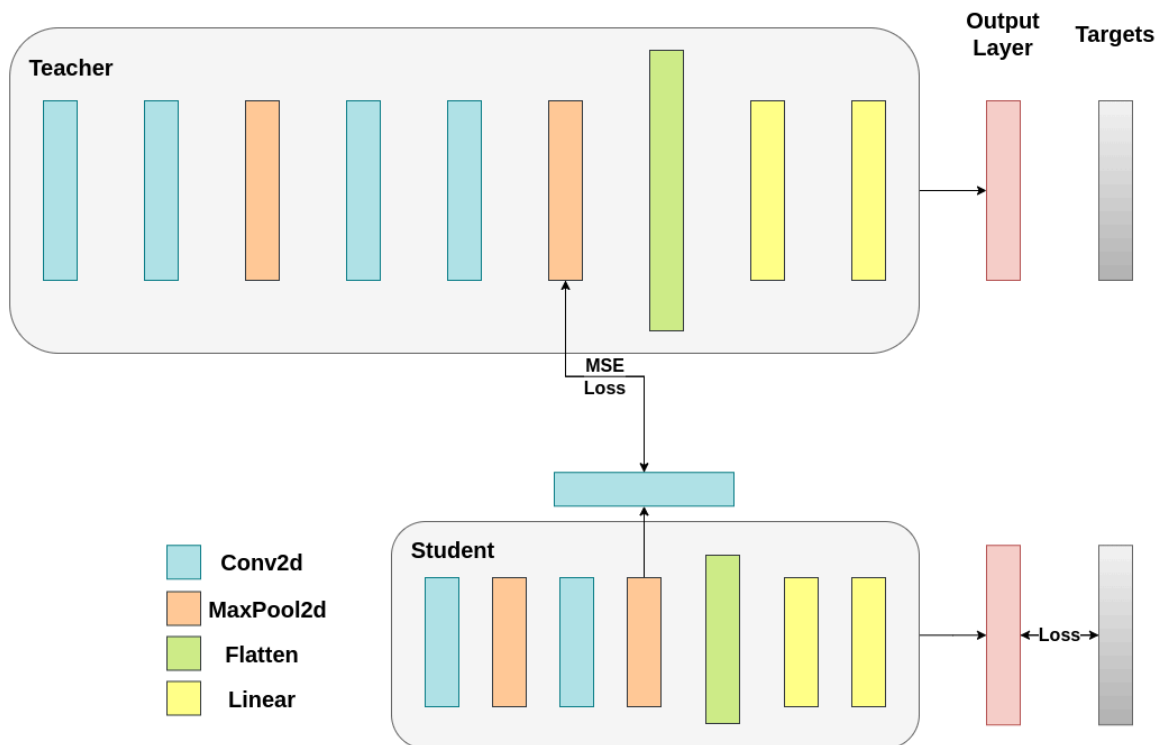
```
In [ ]: convolutional_fe_output_teacher = nn_deep.features(sample_input)
        convolutional_fe_output_student = nn_light.features(sample_input)

# Print their shapes
print("Student's feature extractor output shape: ", convolutional_fe_output_
print("Teacher's feature extractor output shape: ", convolutional_fe_output_
```

```
Student's feature extractor output shape: torch.Size([128, 16, 8, 8])
Teacher's feature extractor output shape: torch.Size([128, 32, 8, 8])
```

We have 32 filters for the teacher and 16 filters for the student. We will include a trainable layer that converts the student's feature map to the shape of the teacher's feature map. In practice, we modify the lightweight class to return the hidden state after an immediate regressor that matches the size of the convolutional feature maps and the teacher class to return the output of the final convolutional layer without pooling or flattening.

```
In [ ]: mo.image(
        src="https://docs.pytorch.org/tutorials/_static/img/knowledge_distillati
    )
```



```
In [ ]: class ModifiedDeepNNRegressor(nn.Module):
    def __init__(self, num_classes=10):
        super(ModifiedDeepNNRegressor, self).__init__()
        self.features = nn.Sequential(
            nn.Conv2d(3, 128, kernel_size=3, padding=1),
            nn.ReLU(),
            nn.Conv2d(128, 64, kernel_size=3, padding=1),
            nn.ReLU(),
            nn.MaxPool2d(kernel_size=2, stride=2),
            nn.Conv2d(64, 64, kernel_size=3, padding=1),
            nn.ReLU(),
            nn.Conv2d(64, 32, kernel_size=3, padding=1),
            nn.ReLU(),
            nn.MaxPool2d(kernel_size=2, stride=2),
        )
        self.classifier = nn.Sequential(
            nn.Linear(2048, 512),
            nn.ReLU(),
            nn.Dropout(0.1),
            nn.Linear(512, num_classes),
        )

    def forward(self, x):
        x = self.features(x)
        conv_feature_map = x
        x = torch.flatten(x, 1)
        x = self.classifier(x)
        return x, conv_feature_map
```

```
In [ ]: class ModifiedLightNNRegressor(nn.Module):
    def __init__(self, num_classes=10):
```

```

super(ModifiedLightNNRegressor, self).__init__()
self.features = nn.Sequential(
    nn.Conv2d(3, 16, kernel_size=3, padding=1),
    nn.ReLU(),
    nn.MaxPool2d(kernel_size=2, stride=2),
    nn.Conv2d(16, 16, kernel_size=3, padding=1),
    nn.ReLU(),
    nn.MaxPool2d(kernel_size=2, padding=2),
)
# Include an extra regressor (in our case linear)
self.regressor = nn.Sequential(
    nn.Conv2d(16, 32, kernel_size=3, padding=1),
)
self.classifier = nn.Sequential(
    nn.Linear(1024, 256), nn.ReLU(), nn.Dropout(0.1), nn.Linear(256,
)

def forward(self, x):
    x = self.features(x)
    regressor_output = self.regressor(x)
    x = torch.flatten(x, 1)
    x = self.classifier(x)
    return x, regressor_output

```

After that, we have to update our train loop again. This time, we extract the regressor output of the student, the feature map of the teacher, we calculate the `MSE` on these tensors (they have the exact same shape so it's properly defined) and we back propagate gradients based on that loss, in addition to the regular cross entropy loss of the classification task.

```

In [ ]: def train_mse_loss(
    teacher,
    student,
    train_loader,
    epochs,
    learning_rate,
    feature_map_weight,
    ce_loss_weight,
    device,
):
    cross_entropy_loss = nn.CrossEntropyLoss()
    mean_squared_error_loss = nn.MSELoss()
    optimizer = optim.Adam(student.parameters(), lr=learning_rate)

    teacher.to(device)
    student.to(device)
    teacher.eval() # Teacher set to evaluation mode
    student.train() # Student to train mode

    for epoch in range(epochs):
        running_loss = 0.0
        for inputs, labels in train_loader:
            inputs, labels = inputs.to(device), labels.to(device)

```



```

optimizer.zero_grad()

with torch.no_grad():
    _, teacher_feature_map = teacher(inputs)

student_logits, student_feature_map = student(inputs)

hidden_rep_loss = mean_squared_error_loss(student_feature_map, t

label_loss = cross_entropy_loss(student_logits, labels)

loss = feature_map_weight * hidden_rep_loss + ce_loss_weight * l

loss.backward()
optimizer.step()

running_loss += loss.item()

print(f"Epoch {epoch + 1}/{epochs}, Loss: {running_loss / len(train_

```

```

In [ ]: modified_nn_light_reg = ModifiedDeepNNRegressor(num_classes=10).to(device)

# We do not have to train the modified deep network from scratch of course,
modified_nn_deep_reg = ModifiedDeepNNRegressor(num_classes=10).to(device)
modified_nn_deep_reg.load_state_dict(nn_deep.state_dict(), strict=True)

# Train and test once again
set_seed(42) # Reset seed for identical initialization
train_mse_loss(
    teacher=modified_nn_deep_reg,
    student=modified_nn_light_reg,
    train_loader=train_loader,
    epochs=10,
    learning_rate=0.001,
    feature_map_weight=0.25,
    ce_loss_weight=0.75,
    device=device,
)

test_accuracy_light_ce_and_mse_loss = test_multiple_outputs(
    modified_nn_light_reg,
    test_loader,
    device,
)

```

```

[W 260707 14:43:24 compiler:352] pytest is not installed, skipping assertion
rewriting
[W 260707 14:43:24 compiler:352] pytest is not installed, skipping assertion
rewriting
[W 260707 14:43:28 compiler:352] pytest is not installed, skipping assertion
rewriting
[W 260707 14:43:28 compiler:352] pytest is not installed, skipping assertion
rewriting
Epoch 1/10, Loss: 1.3354538741623958

```

[W 260707 14:43:43 compiler:352] pytest is not installed, skipping assertion rewriting
[W 260707 14:43:43 compiler:352] pytest is not installed, skipping assertion rewriting
[W 260707 14:43:46 compiler:352] pytest is not installed, skipping assertion rewriting
[W 260707 14:43:47 compiler:352] pytest is not installed, skipping assertion rewriting

Epoch 2/10, Loss: 0.7247414701734968

[W 260707 14:44:02 compiler:352] pytest is not installed, skipping assertion rewriting
[W 260707 14:44:02 compiler:352] pytest is not installed, skipping assertion rewriting
[W 260707 14:44:06 compiler:352] pytest is not installed, skipping assertion rewriting
[W 260707 14:44:06 compiler:352] pytest is not installed, skipping assertion rewriting

Epoch 3/10, Loss: 0.5629884589206228

[W 260707 14:44:21 compiler:352] pytest is not installed, skipping assertion rewriting
[W 260707 14:44:21 compiler:352] pytest is not installed, skipping assertion rewriting
[W 260707 14:44:24 compiler:352] pytest is not installed, skipping assertion rewriting
[W 260707 14:44:24 compiler:352] pytest is not installed, skipping assertion rewriting

Epoch 4/10, Loss: 0.45107074764073657

[W 260707 14:44:40 compiler:352] pytest is not installed, skipping assertion rewriting
[W 260707 14:44:40 compiler:352] pytest is not installed, skipping assertion rewriting
[W 260707 14:44:44 compiler:352] pytest is not installed, skipping assertion rewriting
[W 260707 14:44:44 compiler:352] pytest is not installed, skipping assertion rewriting

Epoch 5/10, Loss: 0.36283701536295665

[W 260707 14:45:00 compiler:352] pytest is not installed, skipping assertion rewriting
[W 260707 14:45:00 compiler:352] pytest is not installed, skipping assertion rewriting
[W 260707 14:45:03 compiler:352] pytest is not installed, skipping assertion rewriting
[W 260707 14:45:03 compiler:352] pytest is not installed, skipping assertion rewriting

Epoch 6/10, Loss: 0.29451709158737643

[W 260707 14:45:18 compiler:352] pytest is not installed, skipping assertion rewriting
[W 260707 14:45:18 compiler:352] pytest is not installed, skipping assertion rewriting
[W 260707 14:45:22 compiler:352] pytest is not installed, skipping assertion rewriting
[W 260707 14:45:22 compiler:352] pytest is not installed, skipping assertion rewriting

Epoch 7/10, Loss: 0.2357697762796641

```
[W 260707 14:45:37 compiler:352] pytest is not installed, skipping assertion rewriting
[W 260707 14:45:37 compiler:352] pytest is not installed, skipping assertion rewriting
[W 260707 14:45:40 compiler:352] pytest is not installed, skipping assertion rewriting
[W 260707 14:45:40 compiler:352] pytest is not installed, skipping assertion rewriting
```

Epoch 8/10, Loss: 0.20208236931458765

```
[W 260707 14:45:56 compiler:352] pytest is not installed, skipping assertion rewriting
[W 260707 14:45:56 compiler:352] pytest is not installed, skipping assertion rewriting
[W 260707 14:45:59 compiler:352] pytest is not installed, skipping assertion rewriting
[W 260707 14:45:59 compiler:352] pytest is not installed, skipping assertion rewriting
```

Epoch 9/10, Loss: 0.1809339991882634

```
[W 260707 14:46:15 compiler:352] pytest is not installed, skipping assertion rewriting
[W 260707 14:46:15 compiler:352] pytest is not installed, skipping assertion rewriting
[W 260707 14:46:18 compiler:352] pytest is not installed, skipping assertion rewriting
[W 260707 14:46:18 compiler:352] pytest is not installed, skipping assertion rewriting
```

Epoch 10/10, Loss: 0.16156046605094923

```
[W 260707 14:46:33 compiler:352] pytest is not installed, skipping assertion rewriting
[W 260707 14:46:33 compiler:352] pytest is not installed, skipping assertion rewriting
[W 260707 14:46:36 compiler:352] pytest is not installed, skipping assertion rewriting
[W 260707 14:46:36 compiler:352] pytest is not installed, skipping assertion rewriting
```

Test accuracy: 73.95%

It is expected that the final method will work better than `CosineLoss` because now we have allowed a trainable layer between the teacher and the student, which gives the student some wiggle room when it comes to learning, rather than pushing the student to copy the teacher's representation. Including the extra network is the idea behind hint-based distillation.

```
In [ ]: print(f"Teacher accuracy: {test_accuracy_deep:.2f}%")
        print(f"Student accuracy without teacher: {test_accuracy_light_ce:.2f}%")
        print(f"Student accuracy with CE + KD: {test_accuracy_light_ce_and_kd:.2f}%")
        print(f"Student accuracy with CE + CosineLoss: {test_accuracy_light_ce_and_c}")
        print(f"Student accuracy with CE + RegressorMSE: {test_accuracy_light_ce_and
```

Teacher accuracy: 74.75%
Student accuracy without teacher: 71.12%
Student accuracy with CE + KD: 71.17%
Student accuracy with CE + CosineLoss: 71.46%
Student accuracy with CE + RegressorMSE: 73.95%